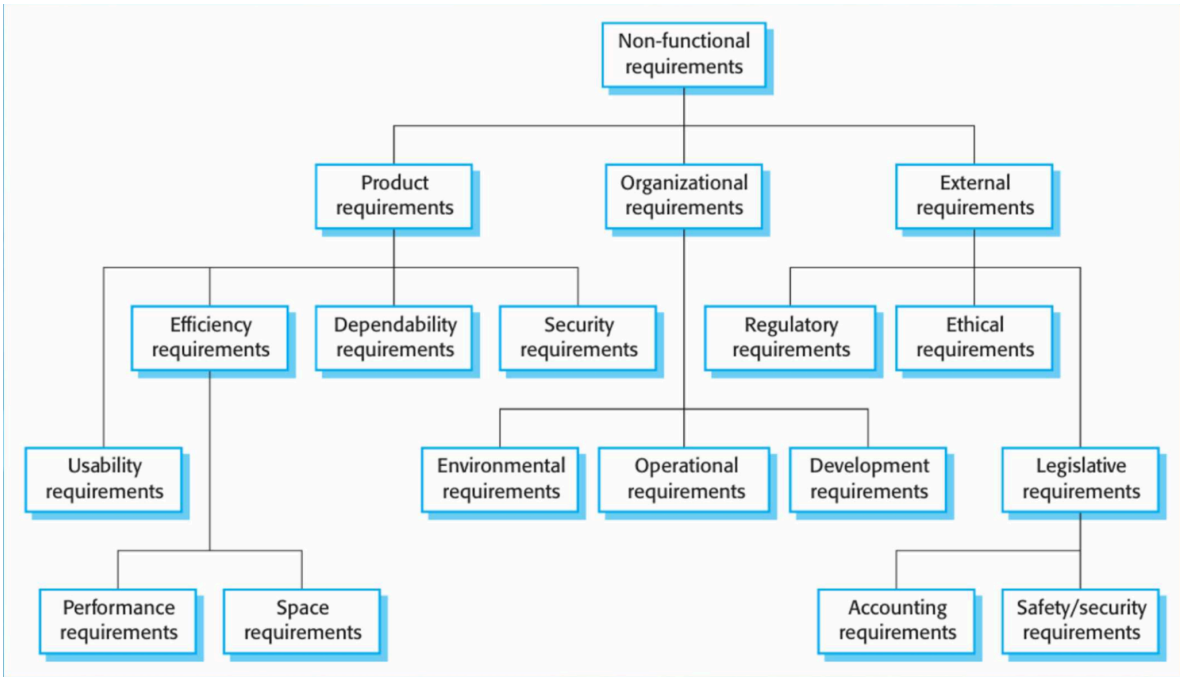
1. Differentiate between Functional & Non-Functional Requirements [U1 C2]

Functional Requirement	Non-Functional Requirement
Describes **what system does**	Describes **how system performs**
Feature-based	Quality/constraint-based
Specific to functions	Applies to whole system
Example: Generate report	Example: Report in 2 seconds

Basis of Difference	Functional Requirements	Non-Functional Requirements
Definition	Describe what the system should do	Describe how the system performs
Focus	System services and features	System quality and constraints
Nature	Behavioral requirements	Quality/constraint requirements
Scope	Specific to particular functions	Apply to entire system
Purpose	Define functionality	Define performance and standards

Measurability	Checked by correct output	Measured using metrics (time, speed, etc.)
Testing	Verified through functional testing	Verified through performance, security, usability testing
Impact	Affect system features	Affect system architecture and design
Documentation	Listed as system functions	Listed as quality attributes
User Visibility	Directly visible to users	Indirectly visible (quality experience)
Failure Impact	System won't perform required task	System works but may be slow/unsafe
Example Statement	"System shall generate report"	"Report shall generate within 2 seconds"
Change Impact	Usually affects specific module	Often affects entire system design
Dependency	Independent feature-based	Often interrelated across system

- **Types of Non-Functional Requirements**



1. **Product Requirements**
    - Performance (response time)
    - Security
    - Reliability
    - Usability

        ✔ Example: System must respond within 2 sec.
2. **Organizational Requirements**
    - Standards
    - Development tools

        ✔ Example: Must be developed using Java.
3. **External Requirements**
    - Legal
    - Regulatory

        ✔ Example: Must follow data protection laws.

2. Agile Method (Brief Detail) [U1 C1]

Agile is an **iterative and incremental** development method.

Key Features:

- Small releases (sprints)
- Customer involvement
- Continuous feedback
- Accepts change

Example:

Instead of building full app at once, release login first, then dashboard, then reports.

3. Agile Development Techniques [U1 C1]

These are practical methods used in Agile.

Techniques:

- Iterative development
- User stories
- Continuous integration
- Test-driven development (TDD)
- Pair programming
- Refactoring
- Daily stand-up meetings

Example:

Write test first → write code → integrate daily.

4. Principles of Agile Method [U1 C1]

Agile Principles

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Embrace change	Expect the system requirements to change, and so design the system to accommodate these changes.
Incremental delivery	The software is developed in increments, with the customer specifying the requirements to be included in each increment.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.
People, not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.

1. Customer satisfaction through early delivery
2. Welcome changing requirements
3. Deliver working software frequently
4. Collaboration between business & developers
5. Face-to-face communication
6. Working software is progress measure
7. Sustainable development
8. Technical excellence

9. Simplicity
10. Self-organizing teams
11. Regular improvement

5. Requirement Analysis & Elicitation Process

🔹 Requirement Elicitation

Gathering requirements from stakeholders.

Methods:

- Interviews
- Observation
- Workshops
- Questionnaires
- Prototyping

🔹 Requirement Analysis

- Remove conflicts
- Check feasibility
- Prioritize requirements

Example:

Customer wants fast system → Analyst defines “2 sec response time”.

6. Extreme Programming (XP) [U1 C1]

An Agile method focusing on engineering practices.

Key Practices:

- Pair programming
- TDD
- Continuous integration
- Small releases
- Refactoring
- On-site customer

Example:

Two programmers write code together and test daily.

---

7. Requirement Engineering Process

Requirement Engineering includes:

1. Feasibility Study
2. Requirement Elicitation
3. Requirement Analysis
4. Requirement Specification
5. Requirement Validation
6. Requirement Management

Example:

Bank system → gather needs → analyze → document → validate → manage changes.

---

8. Scrum Process in Detail [U1 C1]

Scrum works in **Sprints (2–4 weeks)**.

Steps:

1. Product Backlog
2. Sprint Planning
3. Sprint Execution
4. Daily Scrum
5. Sprint Review
6. Sprint Retrospective

Roles:

- Product Owner
- Scrum Master
- Development Team

Example:

Build login feature in Sprint 1, profile feature in Sprint 2.

---

9. Structure of Requirement Document (SRS)

1. Introduction
2. Overall Description
3. Functional Requirements
4. Non-Functional Requirements
5. System Models
6. Appendices

Example:

SRS for Library System includes login, book issue, performance limits, etc.

---

10. What is Requirement & Stakeholders

Requirement:

A condition or capability needed by user.

Stakeholders:

- End users
- Customers
- Developers
- Project manager
- Analysts
- Domain experts
- Regulatory authorities

Example:

Student system → Students, Admin, Developer, College authority.

---

11. Requirement Engineering Process (Spiral View)

Spiral view shows requirement activities in cycles.

Activities in Spiral:

- Elicitation
- Analysis
- Specification
- Validation

After validation → improvements → next cycle.

Example:

Initial requirement → feedback → refine → validate → improve.